

Práctica Nro. 8

Estructuras de control y sentencias

Objetivo: reconocer las diferencias entre las implementaciones de las estructuras de control de los distintos lenguajes

Ejercicio 1: Una sentencia puede ser simple o compuesta, ¿Cuál es la diferencia?

- Sentencia simple: está formada por una única instrucción, realiza una sola acción y no contiene otras sentencias anidadas.
- Sentencia compuesta: agrupa varias instrucciones y se comporta como una sola sentencia. Cada lenguaje utiliza distintos delimitadores para definir el bloque (por ejemplo, begin-end, {}, indentación, etc.).

Ejercicio 2: Analice como C implementa la asignación.

En C la asignación se realiza mediante el operador = [igual]. Las principales características son:

- La expresión de la derecha se evalúa y su resultado se almacena en la variable de la izquierda.
- La asignación es una expresión, por lo que devuelve un valor y puede formar parte de expresiones más complejas.
- Existen operadores compuestos como += ($x+=5 \rightarrow x=x+5$)
- En el caso de asignaciones en condiciones, primero asigna, luego evalúa, y toma como verdadero todo resultado distinto de 0.

Ejercicio 3: ¿Una expresión de asignación puede producir efectos laterales que afecten al resultado final, dependiendo de cómo se evalúe? De ejemplos.

Sí. Un efecto lateral es una modificación del estado del programa producida durante la evaluación de una expresión. Una expresión de asignación puede producir dicho efecto que afecte el resultado final, dependiendo del orden de evaluación de las subexpresiones. Las asignaciones no solo almacenan valores, sino que también pueden modificar variables que se usan en otras partes de la misma expresión.

```
i = 5;
```

```
a = i++ + i;
```

Aquí aparece una modificación de `i` mientras también se utiliza su valor. Dependiendo del orden de evaluación pueden obtenerse resultados distintos.

Ejercicio 4: Qué significa que un lenguaje utilice circuito corto o circuito largo para la evaluación de una expresión. De un ejemplo en el cual por un circuito de error y por el otro no.

Se refiere a como se evalúan las expresiones lógicas en determinados contextos. Son técnicas utilizadas en la evaluación de expresiones booleanas que involucran operadores lógicos como AND u OR.

- **Circuito corto:** significa que los operandos de una expresión lógica se evalúan de izquierda a derecha, pero sólo hasta encontrar el primero que determina el resultado final. Ahí la evaluación se detiene y no se evalúan los operandos restantes. Entonces, en una conjunción (true si ambos términos son verdaderos), evalúa y en caso de que el primer término ya sea falso, termina la evaluación y devuelve false; y en disyunción (true si un término es verdadero) evalúa y en caso de que el primero sea verdadero, termina la evaluación y devuelve true.
- **Circuito largo:** en este caso todos los operandos se evalúan sin importar si el primero ya determina el resultado final.

Ejercicio 5: ¿Qué regla define Delphi, Ada y C para la asociación del else con el if correspondiente? ¿Cómo lo maneja Python?

En C y Delphi, cuando existen varios if anidados, el else se asocia automáticamente con el if abierto más cercano. Esta regla elimina la ambigüedad sintáctica, aunque puede dificultar la lectura del código cuando no se utilizan bloques claramente delimitados.

En Ada, la ambigüedad se evita mediante el uso de delimitadores explícitos para cerrar cada estructura condicional (end if), por lo que queda perfectamente definido a qué if pertenece cada else.

En Python, el problema se resuelve mediante la indentación obligatoria. Los bloques quedan determinados por la sangría, lo que elimina la posibilidad de interpretar un else asociado a un if diferente. Además, incorpora la cláusula elif para representar múltiples alternativas de forma más legible.

Ejercicio 6: ¿Cuál es la construcción para expresar múltiple selección que implementa C?
¿Trabaja de la misma manera que la de Pascal, ADA o Python?

En C, la selección múltiple se implementa mediante la sentencia switch. Cada alternativa se expresa mediante una cláusula case y existe una cláusula opcional default para los casos no contemplados. Una característica particular es que, si no se utiliza break, la ejecución continúa en los casos siguientes (fenómeno conocido como fall-through).

En Pascal, la selección múltiple se realiza mediante case ... of. A diferencia de C, una vez seleccionada una alternativa no se continúa ejecutando las siguientes. Además, la cláusula else es opcional, lo que puede generar situaciones inseguras si no se contemplan todos los valores posibles.

En Ada, la construcción es case ... is junto con cláusulas when. Es más estricta que las anteriores, ya que obliga a cubrir todos los valores posibles de la expresión o a incluir una cláusula when others; de lo contrario, el compilador produce un error.

En Python, según el material, la selección múltiple se resuelve mediante encadenamientos de if, elif y else, utilizando la indentación para delimitar los bloques y mejorar la legibilidad.

Ejercicio 7: Sea el siguiente código

<pre>..... var i, z:integer; Procedure A; begin i:= i +1; end; begin z:=5;</pre>	<pre>for i:=1..5 do begin z:=z*5; A; z:=z + i; end; end;</pre>
--	--

- Analice en las versiones estándar de ADA y Pascal, si este código puede llegar a traer problemas. Justifique la respuesta.

Sí, puede traer problemas porque el procedimiento A modifica la variable i, que es la variable de control del bucle for.

En Pascal estándar, no está permitido modificar la variable de control dentro del cuerpo del for. El estándar establece que no deben modificarse ni la variable de control ni los límites del ciclo. Alterar i dentro del procedimiento A genera un comportamiento incorrecto o no permitido.

En Ada, la variable de control del for es implícita y su valor está controlado por el propio bucle. No puede modificarse dentro del ciclo. Por lo tanto, intentar cambiarla desde un procedimiento también constituye una situación inválida y el programa no compilaría. En ambos lenguajes el problema es que el programador intenta alterar una variable cuyo valor debe ser administrado exclusivamente por la estructura iterativa, pudiendo afectar el comportamiento y la corrección del ciclo.

Ejercicio 8: Sea el siguiente código en Pascal

```
var puntos: integer;
begin
...
case puntos
1..5: write("No puede continuar");
10:write("Trabajo terminado")
end;
..
```

Analice, si esto mismo, con la sintaxis correspondiente, puede trasladarse así a los lenguajes ADA, C. ¿Provocarí error en algún caso? Diga cómo debería hacerse en cada lenguaje y explique el por qué. Codifíquelo.

El código es en Pascal: no genera error pues el lenguaje permite usar rangos dentro del case (por eso vale el 1..5) y además no necesito cubrir todos los casos, si sale un caso que no está contemplado entonces no se ejecuta ninguna rama.

En el caso de ADA, necesito cubrir si o si todos los casos, por eso es necesario usar el *when others*, caso contrario se genera un error en compilación.

En el caso de C, puedo tener casos sin cubrir (eso funciona como en Pascal) pero no puedo definir el case usando rangos porque tendría un error de sintaxis, en este caso en lugar de usar case uso *switch*. Los códigos quedarían:

ADA	C
<pre>case Puntos is when 1..5 => Put_Line("No continuar") when 10 => Put_Line("Trabajo terminado") when others null; end case;</pre>	<pre>switch (puntos) { case 1: ... case 5: print("No continuar"); break; case 10: print("Trabajo terminado"); break; default: break; }</pre>

Ejercicio 9: Qué diferencia existe entre el generador YIELD de Python y el return de una función. De un ejemplo donde sería útil utilizarlo.

La principal diferencia es que *return* finaliza la ejecución de la función y devuelve un único resultado, mientras que *yield* suspende temporalmente la ejecución, devuelve un valor y conserva el estado de la función para continuar más adelante.

Las funciones que utilizan *yield* se denominan generadores. Permiten producir una secuencia de valores de manera gradual, sin necesidad de almacenarlos todos en memoria al mismo tiempo.

El uso de *yield* resulta especialmente útil cuando se trabaja con grandes volúmenes de datos, archivos extensos o secuencias potencialmente infinitas, ya que permite obtener los elementos uno a uno a medida que se necesitan.

Ejercicio 10: Describa brevemente la instrucción map en javascript y sus alternativas.

La función *map()* de JavaScript permite recorrer los elementos de un arreglo y generar un nuevo arreglo aplicando una transformación a cada elemento. *map()* suele elegirse cuando se desea transformar cada elemento de una colección manteniendo la estructura del arreglo resultante.

Sus características principales son:

- Recorre todos los elementos del arreglo.
- No modifica el arreglo original.
- Devuelve un nuevo arreglo con los resultados de la transformación.
- Mantiene la misma cantidad de elementos que el arreglo original.

Algunas alternativas son:

- *for*: recorrido tradicional mediante índices.
- *forEach()*: recorre todos los elementos, pero no devuelve un nuevo arreglo.
- *filter()*: devuelve únicamente los elementos que cumplen una condición.
- *reduce()*: combina todos los elementos para obtener un único resultado.
- *for...of*: permite recorrer colecciones de forma más legible.

Ejercicio 11: Determine si el lenguaje que utiliza frecuentemente implementa instrucciones para el manejo de espacio de nombres. Mencione brevemente qué significa este concepto y enuncie la forma en que su lenguaje lo implementa. Enuncie las características más importantes de este concepto en lenguajes como PHP o Python.

Un espacio de nombres (namespace) es un mecanismo que permite organizar identificadores (variables, funciones, clases, módulos, etc.) para evitar conflictos entre nombres iguales y mejorar la organización de programas grandes.

Tomando como ejemplo Python, sí implementa este concepto mediante módulos y paquetes.

Las características principales son:

- Cada módulo posee su propio espacio de nombres.
- Los nombres definidos en un módulo no interfieren con los de otros módulos.
- Se pueden importar módulos completos o elementos específicos mediante *import* y *from ... import*.
- Los paquetes permiten agrupar varios módulos relacionados.

En PHP, los espacios de nombres se implementan explícitamente mediante la palabra reservada *namespace*.

Características importantes en PHP:

- Permiten organizar clases, interfaces, funciones y constantes.
- Evitan conflictos entre bibliotecas desarrolladas por distintos programadores.
- Se utilizan junto con la instrucción *use* para importar elementos de otros espacios de nombres.
- Son ampliamente utilizados en frameworks y aplicaciones grandes.

Tanto en Python como en PHP, el objetivo principal de los espacios de nombres es evitar colisiones de identificadores, favorecer la modularidad y facilitar el mantenimiento del software.